# C++ Question Bank

63 multiple-choice questions (Core 20, Easy 8, Medium 18, Hard 17) and 8 coding questions. The correct answer is shown under each question.

## MCQs - Core (20)

**1. Which of these is NOT one of the four pillars of object-oriented programming?**

**A)** Encapsulation
**B)** Inheritance
**C)** Compilation
**D)** Polymorphism

**Answer: C) Compilation**

**2. What is the difference between a struct and a class in C++?**

**A)** A struct cannot have member functions
**B)** Members and base classes are public by default in a struct and private by default in a class
**C)** A struct cannot be inherited
**D)** There is no difference at all

**Answer: B) Members and base classes are public by default in a struct and private by default in a class**

**3. What is the time complexity of accessing v[i] in a std::vector?**

**A)** O(1)
**B)** O(log n)
**C)** O(n)
**D)** O(n log n)

**Answer: A) O(1)**

**4. Which STL container gives average O(1) lookup by key (without ordering the keys)?**

**A)** std::map
**B)** std::unordered_map
**C)** std::list
**D)** std::vector

**Answer: B) std::unordered_map**

**5. What is the output?**

```
vector<int> v = {4, 1, 3};
sort(v.begin(), v.end());
for (int x : v) cout << x;
```

**A)** 413
**B)** 134
**C)** 431
**D)** 314

**Answer: B) 134**

**6. What is the output?**

```
queue<int> q;
q.push(1); q.push(2); q.push(3);
q.pop();
cout << q.front() << q.back();
```

**A)** 23
**B)** 12
**C)** 32
**D)** 13

**Answer: A) 23**

**7. Which is the correct way to release an array allocated with int *a = new int[10]; ?**

**A)** delete a;
**B)** delete[] a;
**C)** free(a);
**D)** No release is needed

**Answer: B) delete[] a;**

**8. What is encapsulation?**

**A)** Deriving a class from another
**B)** Bundling data with the methods that operate on it and restricting direct access to the data
**C)** Having many functions with the same name
**D)** Allocating objects on the heap

**Answer: B) Bundling data with the methods that operate on it and restricting direct access to the data**

**9. What is the output?**

```
string s = "coding";
cout << s.substr(1, 3) << s.length();
```

**A)** odi6
**B)** cod6
**C)** odin6
**D)** odi5

**Answer: A) odi6**

**10. Which feature lets a derived class provide its own implementation of a base class virtual function?**

**A)** Overloading
**B)** Overriding
**C)** Templates
**D)** Friend functions

**Answer: B) Overriding**

**11. What is the output?**

```
int a[] = {3, 7, 2, 9};
int mx = a[0];
for (int x : a) if (x > mx) mx = x;
cout << mx;
```

**A)** 3
**B)** 7
**C)** 9
**D)** 2

**Answer: C) 9**

**12. In a binary search tree, where is a value smaller than the root stored?**

**A)** In the right subtree
**B)** In the left subtree
**C)** At a leaf only
**D)** Anywhere

**Answer: B) In the left subtree**

**13. What does the this pointer refer to inside a non-static member function?**

**A)** The class itself
**B)** The object on which the function was called
**C)** The base class
**D)** The last created object

**Answer: B) The object on which the function was called**

**14. Which container is best when you need fast insertion and removal at both the front and the back?**

**A)** std::vector
**B)** std::deque
**C)** std::set
**D)** std::array

**Answer: B) std::deque**

**15. What is the output?**

```
int x = 5;
int *p = &x;
*p = *p * 2;
cout << x;
```

**A)** 5
**B)** 10
**C)** Address of x
**D)** 25

**Answer: B) 10**

**16. What is a pure virtual function?**

**A)** A function that cannot be overridden
**B)** A virtual function declared with = 0 that makes the class abstract
**C)** A static function
**D)** An inline function with no body

**Answer: B) A virtual function declared with = 0 that makes the class abstract**

**17. What is the output?**

```
map<char, int> freq;
for (char c : string("banana")) freq[c]++;
cout << freq['a'] << freq['n'] << freq.size();
```

**A)** 323
**B)** 322
**C)** 213
**D)** 331

**Answer: A) 323**

**18. Function overloading is resolved by:**

**A)** The return type only
**B)** The number and types of the parameters
**C)** The function body
**D)** The name of the parameters

**Answer: B) The number and types of the parameters**

**19. What is the worst-case time complexity of searching for a value in an unsorted std::vector?**

**A)** O(1)
**B)** O(log n)
**C)** O(n)
**D)** O(n^2)

**Answer: C) O(n)**

**20. What is the output?**

```
class Box {
    int w;
public:
    Box(int x) : w(x) {}
    int area() { return w * w; }
};
...
Box b(4);
cout << b.area();
```

**A)** 4
**B)** 8
**C)** 16
**D)** Compilation error: w is private

**Answer: C) 16**

## MCQs - Easy (8)

**21. Which C++ feature allows a function to have the same name but different parameter lists?**

**A)** Function overriding
**B)** Function overloading
**C)** Templates
**D)** Inline functions

**Answer: B) Function overloading**

**22. What is the default access specifier for members of a class in C++?**

**A)** public
**B)** protected
**C)** private
**D)** friend

**Answer: C) private**

**23. What is the output?**

```
int x = 10;
int &r = x;
r = 20;
std::cout << x;
```

**A)** 10
**B)** 20
**C)** Garbage value
**D)** Compilation error

**Answer: B) 20**

**24. Which STL container stores unique elements in sorted order?**

**A)** std::vector
**B)** std::list
**C)** std::set
**D)** std::deque

**Answer: C) std::set**

**25. Which operator cannot be overloaded in C++?**

**A)** +
**B)** []
**C)** ::
**D)** ==

**Answer: C) ::**

**26. What does std::vector<int> v(5, 2); create?**

**A)** A vector with elements 5 and 2
**B)** A vector of 5 elements, each equal to 2
**C)** A vector of 2 elements, each equal to 5
**D)** A compilation error

**Answer: B) A vector of 5 elements, each equal to 2**

**27. What is the output?**

```
class A { public: void show(){ std::cout << "A"; } };
class B : public A { public: void show(){ std::cout << "B"; } };
A *p = new B(); p->show();
```

**A)** A
**B)** B
**C)** AB
**D)** Compilation error

**Answer: A) A**

**28. What does const int *p mean?**

**A)** p is a constant pointer to int
**B)** p is a pointer to a constant int (value cannot be changed through p)
**C)** Both pointer and value are constant
**D)** Invalid declaration

**Answer: B) p is a pointer to a constant int (value cannot be changed through p)**

## MCQs - Medium (18)

**29. What is the output?**

```
struct A { A() { cout << "A"; } ~A() { cout << "a"; } };
struct B : A { B() { cout << "B"; } ~B() { cout << "b"; } };
int main() { B obj; }
```

**A)** ABab
**B)** ABba
**C)** BAba
**D)** BAab

**Answer: B) ABba**

**30. What is the output?**

```
class C { public: static int n; C() { n++; } };
int C::n = 0;
int main() { C a, b[3]; cout << C::n; }
```

**A)** 1
**B)** 3
**C)** 4
**D)** 0

**Answer: C) 4**

**31. What is the output?**

```
void f(int x)  { x = 10; }
void g(int &x) { x = 20; }
int main() { int a = 1; f(a); cout << a; g(a); cout << a; }
```

**A)** 1020
**B)** 120
**C)** 11
**D)** 1010

**Answer: B) 120**

**32. What is the output?**

```
vector<int> v = {1, 2, 3};
v.push_back(4);
v.pop_back();
v.insert(v.begin(), 0);
cout << v.size() << v[0];
```

**A)** 30
**B)** 40
**C)** 41
**D)** 50

**Answer: B) 40**

**33. What is the output?**

```
map<string, int> m;
m["b"] = 2; m["a"] = 1; m["c"] = 3;
for (auto &p : m) cout << p.first;
```

**A)** bac
**B)** abc
**C)** cba
**D)** Unspecified order

**Answer: B) abc**

**34. What is the output?**

```
int f(int a, int b = 2, int c = 3) { return a + b + c; }
...
cout << f(1) << " " << f(1, 1);
```

**A)** 6 5
**B)** 6 6
**C)** 1 2
**D)** Compilation error

**Answer: A) 6 5**

### 35. What is the output?

```
vector<int> v{1, 2, 3};
for (auto x : v)  x *= 2;
cout << v[0];
for (auto &x : v) x *= 2;
cout << v[0];
```

**A)** 22
**B)** 12
**C)** 11
**D)** 24

**Answer: B) 12**

### 36. What is the output?

```
string s = "hello";
s += " world";
cout << s.find("o") << " " << s.size();
```

**A)** 4 11
**B)** 5 11
**C)** 4 10
**D)** 7 11

**Answer: A) 4 11**

### 37. What is the output?

```
try { throw 5; }
catch (double) { cout << "d"; }
catch (int)    { cout << "i"; }
catch (...)    { cout << "all"; }
```

**A)** d
**B)** i
**C)** all
**D)** iall

**Answer: B) i**

### 38. What is the output?

```
int x = 10;
int main() { int x = 20; cout << x << ::x; }
```

**A)** 2020
**B)** 1010
**C)** 2010
**D)** 1020

**Answer: C) 2010**

### 39. What is the output?

```
set<int> s = {3, 1, 3, 2, 1};
cout << s.size() << *s.begin();
```

**A)** 51
**B)** 31
**C)** 33
**D)** 53

**Answer: B) 31**

**40. What is the output?**

```
priority_queue<int> pq;
pq.push(3); pq.push(1); pq.push(4);
pq.pop();
cout << pq.top();
```

**A)** 1
**B)** 3
**C)** 4
**D)** Compilation error

**Answer: B) 3**

**41. What is the output?**

```
class E {};
cout << sizeof(E);
```

**A)** 0
**B)** 1
**C)** 4
**D)** 8

**Answer: B) 1**

**42. What happens?**

```
class A {
  int v = 0;
public:
  void set(int x) { v = x; }
};
int main() { const A a; a.set(5); }
```

**A)** It compiles and a.v becomes 5
**B)** Compilation error: a non-const member function cannot be called on a const object
**C)** Runtime error
**D)** It compiles, but the assignment is ignored

**Answer: B) Compilation error: a non-const member function cannot be called on a const object**

**43. What is the output?**

```
struct Shape { virtual void draw() = 0; };
int main() { Shape s; }
```

**A)** Nothing is printed
**B)** Compilation error: cannot declare a variable of abstract type
**C)** Runtime error: pure virtual call
**D)** Linker error

**Answer: B) Compilation error: cannot declare a variable of abstract type**

**44. Why is virtual inheritance used, e.g. class B : virtual public A ?**

**A)** To make all methods of A virtual
**B)** To make sure only one shared A sub-object exists in a diamond hierarchy
**C)** To prevent A from being inherited again
**D)** To allow A's private members to be accessed

**Answer: B) To make sure only one shared A sub-object exists in a diamond hierarchy**

### 45. What is the output?

```
void f(int)    { cout << "int "; }
void f(double) { cout << "double "; }
...
f('a');
f(2.5f);
```

**A)** int double
**B)** double double
**C)** int int
**D)** Compilation error (ambiguous)

**Answer: A) int double**

### 46. What is the output?

```
stack<int> s;
s.push(1); s.push(2); s.push(3);
s.pop();
cout << s.top() << s.size();
```

**A)** 22
**B)** 32
**C)** 12
**D)** 23

**Answer: A) 22**

## MCQs - Hard (17)

### 47. What is the output?

```
struct A {
  A() {}
  A(const A &) { cout << "C"; }
};
void f(A a) {}
int main() { A x; f(x); A y = x; A z; z = x; }
```

**A)** CCC
**B)** CC
**C)** C
**D)** CCCC

**Answer: B) CC**

### 48. What is the output?

```
struct B { virtual ~B() { cout << "~B"; } };
struct D : B { ~D() { cout << "~D"; } };
int main() { B *p = new D; delete p; }
```

**A)** ~B
**B)** ~D
**C)** ~D~B
**D)** ~B~D

**Answer: C) ~D~B**

### 49. What is the output?

```
struct B { B() { f(); } virtual void f() { cout << "B"; } };
struct D : B { void f() override { cout << "D"; } };
int main() { D d; }
```

**A)** B
**B)** D
**C)** BD
**D)** Undefined behaviour

**Answer: A) B**

### 50. What is the output?

```
struct Base    { virtual void print() { cout << "Base"; } };
struct Derived : Base { void print() override { cout << "Derived"; } };
void show(Base b) { b.print(); }
int main() { Derived d; show(d); }
```

**A)** Base
**B)** Derived
**C)** BaseDerived
**D)** Compilation error

**Answer: A) Base**

### 51. What is the output?

```
int x = 1;
auto f = [x]  { return x; };
auto g = [&x] { return x; };
x = 5;
cout << f() << g();
```

**A)** 11
**B)** 55
**C)** 15
**D)** 51

**Answer: C) 15**

### 52. What happens?

```
unique_ptr<int> a = make_unique<int>(5);
unique_ptr<int> b = a;
```

**A)** b and a both point to 5
**B)** Ownership is transferred to b, and a becomes null
**C)** Compilation error: unique_ptr cannot be copied
**D)** Runtime error (double delete)

**Answer: C) Compilation error: unique_ptr cannot be copied**

### 53. What is the output?

```
auto a = make_shared<int>(5);
auto b = a;
{ auto c = a; }
cout << a.use_count();
```

**A)** 1
**B)** 2
**C)** 3
**D)** 0

**Answer: B) 2**

**54. What is the output?**

```
struct B { virtual ~B() {} };
struct D : B {};
int main() {
  B *b = new B;
  D *d = dynamic_cast<D *>(b);
  cout << (d == nullptr);
}
```

**A)** 0
**B)** 1
**C)** Compilation error
**D)** Throws std::bad_cast

**Answer: B) 1**

**55. What is the output?**

```
template <typename T> void f(T)  { cout << "T"; }
template <> void f<int>(int)      { cout << "int"; }
int main() { f(1); f(1.0); f('c'); }
```

**A)** intintint
**B)** intTT
**C)** TTT
**D)** intTint

**Answer: B) intTT**

**56. What is the output?**

```
struct E {};
struct F : E {};
int main() {
  try { throw F(); }
  catch (E &) { cout << "E"; }
  catch (F &) { cout << "F"; }
}
```

**A)** F
**B)** E
**C)** EF
**D)** Compilation error

**Answer: B) E**

**57. What is the output?**

```
struct B { void f(int)    { cout << "B"; } };
struct D : B { void f(double) { cout << "D"; } };
int main() { D d; d.f(1); }
```

**A)** B
**B)** D
**C)** Compilation error (ambiguous)
**D)** BD

**Answer: B) D**

**58. What is the output?**

```
map<int, int> m;
if (m[5] == 0) cout << m.size();
```

**A)** 0
**B)** 1
**C)** Nothing is printed
**D)** Throws std::out_of_range

**Answer: B) 1**

**59. Consider the code below. What is the value of a after S s; ?**

```
struct S {
  int a, b;
  S() : b(5), a(b + 1) {}
};
```

**A)** 6
**B)** 1
**C)** Indeterminate - members are initialised in declaration order, so b is not yet set when a is initialised
**D)** Compilation error

**Answer: C) Indeterminate - members are initialised in declaration order, so b is not yet set when a is initialised**

**60. What is the risk in this code?**

```
vector<int> v = {1, 2, 3};
int &first = v[0];
v.push_back(4);
cout << first;
```

**A)** None - it prints 1
**B)** push_back may reallocate the storage, leaving `first` a dangling reference (undefined behaviour)
**C)** Compilation error
**D)** It always prints 4

**Answer: B) push_back may reallocate the storage, leaving `first` a dangling reference (undefined behaviour)**

**61. What is the output?**

```
int a = 5;
int &r = a;
int *p = &r;
*p = 7;
r++;
cout << a;
```

**A)** 5
**B)** 7
**C)** 8
**D)** 6

**Answer: C) 8**

**62. What is the output?**

```
struct P { int v; P(int x) : v(x) {}
  P operator+(const P &o) const { return P(v * o.v); } };
int main() { P a(2), b(3), c(4); cout << (a + b + c).v; }
```

**A)** 9
**B)** 24
**C)** 10
**D)** Compilation error

**Answer: B) 24**

**63. What is the output?**

```
int i = 10;
const int *p = &i;
int *const q = &i;
*q = 20;
cout << *p;
```

**A)** 10
**B)** 20
**C)** Compilation error at *q = 20
**D)** Undefined behaviour

**Answer: B) 20**

# Coding Questions (C++)

Every program reads from standard input and writes to standard output. Marks are awarded in proportion to hidden test cases passed.

### C1. Centered Star Pyramid (Paper A)

Read the number of levels and print a centered star pyramid (example for levels = 5):

Example (levels = 5):

```
    *
   ***
  *****
 *******
*********
```

**Approach:** Row i (0-based): print (levels-1-i) spaces followed by (2*i+1) stars.

### C2. First Standalone Character (Paper A)

Read a string of lowercase letters and find the first character that does not belong to any consecutive repeating group. A character belongs to a repeating group if the same character appears immediately before or after it. Print the first standalone character; if none exists print "No Character Found".

| Sample input | Sample output |
|---|---|
| `aabbccdefggd` | `d` |

**Approach:** Scan the string grouping consecutive identical characters; print the first character whose group length is 1, otherwise print "No Character Found".

### C3. Missing Number in 1..N (Paper A)

The first line of input contains N. The second line contains N-1 distinct integers from 1 to N with exactly one number missing. Print the missing number.

| Sample input | Sample output |
|---|---|
| `5`<br>`1 2 3 5` | `4` |

**Approach:** Expected sum = N*(N+1)/2; subtract the sum of the given numbers (or XOR all values 1..N with the array).

### C4. Prime Number Triangle (Paper A)

Read N and print a right-angled triangle of consecutive prime numbers with N rows, where row i contains i primes separated by single spaces.

| Sample input | Sample output |
|---|---|
| `5` | `2`<br>`3 5`<br>`7 11 13`<br>`17 19 23 29`<br>`31 37 41 43 47` |

**Approach:** Generate primes on demand with a helper isPrime(); for row i print the next i primes.

### C5. Pascal's Triangle (Paper B)

Write a program that reads an integer n from standard input and prints Pascal's Triangle with n rows. Print the numbers of each row separated by a single space, one row per line (left-aligned).

| Sample input | Sample output |
|---|---|
| `4` | `1`<br>`1 1`<br>`1 2 1`<br>`1 3 3 1` |

**Approach:** Each row starts and ends with 1; every interior value is the sum of the two values above it (row[i-1][j-1] + row[i-1][j]).

### C6. Left-aligned Star Pyramid (Paper B)

Read n and print a left-aligned, increasing star pyramid with n rows where row i has 2*i-1 stars.

| Sample input | Sample output |
|---|---|
| `5` | `*`<br>`***`<br>`*****`<br>`*******`<br>`*********` |

**Approach:** Row i (1-based) prints 2*i-1 stars with no leading spaces.

### C7. First Non-Repeating Character (Paper B)

Read a string and print the first character that appears exactly once in the whole string (left-to-right order). If every character repeats, print "None".

| Sample input | Sample output |
|---|---|
| `aabbcdd` | `c` |

**Approach:** Count the frequency of each character, then scan left to right and print the first character with count 1.

### C8. Top K Frequent Elements (Paper B)

The first line contains the integers of an array separated by spaces. The second line contains k. Print the k most frequent elements in descending order of frequency, separated by spaces (if two elements have the same frequency, print the smaller value first).

| Sample input | Sample output |
|---|---|
| `1 1 1 2 2 3 4 4 4 4`<br>`2` | `4 1` |

**Approach:** Count frequencies in a map, sort keys by (frequency desc, value asc) and print the first k.

Paper A: C1-C4  Paper B: C5-C8